

Guide Utilisateur - Thumalien

Système de Détection de Fake News Bilingue FR/EN

1. Présentation

Thumalien est un système d'analyse automatisée de contenus textuels pour la détection de fake news. Il traite les posts publiés sur le réseau social Bluesky en français et en anglais, et fournit pour chaque texte un score de crédibilité, une émotion dominante et une classification fiable/suspect.

Le système repose sur un pipeline NLP bilingue V9 combinant un filtre fait/opinion (Stage 1) avec un ensemble hybride TF-IDF V5 + Style V6 + CamemBERT (V8) + explicabilité SHAP. Plus de 245 000 posts ont été collectés (collecte continue) et 500 ont été annotés manuellement par 2 annotateurs ($\kappa = 0.498$). Le pipeline V9 réduit les faux positifs de 67% par rapport à V5 seul sur le gold standard consensus.

2. Prérequis

- Python 3.13+
- pip (gestionnaire de paquets)
- MongoDB (optionnel - le dashboard fonctionne en mode démo sans base de données)
- Environ 2 Go d'espace disque (modèles + données)

3. Installation

```
# Cloner le depot
git clone https://github.com/azelbanks/projet_etude.git
cd projet_etude

# Installer les dependances
pip install -r requirements.txt

# Verifier l'installation
python -c "from src.pipeline.expert_detector import ExpertFakeNewsDetector; print('OK')"
```

4. Structure du projet

```
projet_etude/
|-- dashboard/
|   |-- app.py           # Dashboard Streamlit (interface principale)
|-- data/
|   |-- training/        # Datasets d'entraînement (non versionnés)
|   |   |-- Fake.csv     # ISOT Fake News (anglais)
|   |   |-- True.csv     # ISOT True News (anglais)
```

```

| | |-- kaggle_fr/          # Kaggle FrenchFakeNewsDetector
| | |-- train.csv          # Émotions EN (entraînement)
| | |-- training.csv       # Émotions FR (entraînement)
|-- docs/
| |-- guide_utilisateur.md # Ce fichier
| |-- architecture.png     # Schéma d'architecture du pipeline
| |-- gantt_planning.png   # Diagramme de Gantt rétrospectif
|-- models/
| |-- model_expert.pkl      # Modèle LogReg bilingue (V1.5)
| |-- tfidf_expert.pkl      # Vectoriseur TF-IDF (V1.5)
| |-- model_expert_v5.pkl   # Modèle LogReg bilingue V5
| |-- tfidf_expert_v5.pkl   # Vectoriseur TF-IDF V5 (30K features)
| |-- metrics_expert.pkl    # Métriques d'entraînement
| |-- model_style_v6.joblib # Modèle style-only V6
| |-- model_hybrid_v7.joblib # Méta-learner hybride V7
| |-- model_hybrid_v8.joblib # Méta-learner V8 (+CamemBERT)
| |-- stage1_fact_opinion.joblib # Classifieur fait/opinion (V9)
| |-- emotion_bilingual.pt  # MLP PyTorch (7 émotions)
| |-- emotion_vocab_bilingual.pickle
| |-- emotion_label_encoder_bilingual.pickle
|-- notebooks/
| |-- 00_Audit_Qualite_Donnees.ipynb
| |-- 01_Exploration_Bluesky.ipynb
| |-- 02_Analyse_Emotions_MLP.ipynb
| |-- 03_Mise_a_jour_Quotidienne.ipynb
| |-- 04_Modele_Avance_RoBERTa.ipynb
| |-- 05_Detection_Expert_Bilingue.ipynb
| |-- 06_Documentation_Technique.ipynb
| |-- 09-24                  # Scripts d'entraînement V3-V7
| |-- 25-27                  # V8 CamemBERT, self-training, pipeline 2 étapes
|-- src/
| |-- pipeline/
| | |-- expert_detector.py # Pipeline complet (classes principales)
| |-- collection/
| | |-- collect_bluesky.py # Collecte AT Protocol
| |-- app/
| | |-- main.py            # Point d'entrée applicatif
|-- .streamlit/
| |-- config.toml          # Thème dark + config serveur
|-- docker-compose.yml
|-- dockerfile
|-- requirements.txt
|-- emissions.csv          # Bilan carbone CodeCarbon

```

5. Lancer le dashboard

```

# Depuis la racine du projet
streamlit run dashboard/app.py

```

Le dashboard s'ouvre automatiquement dans le navigateur à l'adresse <http://localhost:8501>.

Mode démo : si MongoDB n'est pas connecté, le dashboard affiche des données d'exemple (15 posts FR/EN) avec un bandeau informatif.

6. Pages du dashboard

6.1. Vue Globale

La page d'accueil présente une vision synthétique de l'ensemble des posts analysés :

- Métriques clés : nombre total de posts, pourcentage de posts fiables, score de crédibilité moyen, répartition FR/EN.
- Profil émotionnel : radar chart des 7 émotions moyennes sur l'ensemble du corpus (colère, dégoût, joie, neutre, peur, surprise, tristesse).
- Fiabilité par langue : diagramme en barres horizontales comparant les posts fiables et suspects pour le français et l'anglais.
- Tableau des posts : liste des derniers posts analysés avec leur texte (tronqué), langue, label et score de crédibilité.

6.2. Analyse en temps réel

Cette page permet d'analyser un texte en temps réel :

- Collez un texte (article, post Bluesky, tweet, ou tout texte FR/EN) dans la zone de saisie.
- Cliquez sur le bouton Analyser.
- Le système retourne :
 - Un score de crédibilité (jauge 0 à 1) : 0 = probablement faux, 1 = probablement fiable.
 - Un verdict : FIABLE (vert) ou SUSPECT (rouge).
 - L'émotion dominante détectée avec sa probabilité.
 - Un radar chart détaillé des 7 probabilités émotionnelles.
 - La langue détectée automatiquement (FR ou EN).
- La section V8 Hybride affiche :
 - Le type de post détecté (factuel ou opinion) par le filtre Stage 1
 - 5 métriques : score V5 (TF-IDF), score V6 (Style), score CamemBERT (FR), score V8 (Hybride), désaccord V5/V6.
 - Un graphique SHAP montrant la contribution de chaque feature stylistique.
 - Le détail des 35 features avec leur valeur SHAP.

Interprétation du score :

- Score > 0.7 : le texte présente des caractéristiques de contenu fiable.
- Score entre 0.4 et 0.7 : zone d'incertitude, vérification manuelle recommandée.
- Score < 0.4 : le texte présente des marqueurs de désinformation.

> Avertissement : le score est un indicateur probabiliste basé sur des patterns statistiques. Il ne constitue pas une vérification factuelle et ne doit pas être utilisé comme seul critère de décision.

6.3. Métriques & Transparence

Cette page fournit les indicateurs de performance et de conformité :

- Ablation study : tableau et graphique des F1-scores pour les 7 conditions expérimentales testées (EN seul, FR seul, bilingue, bilingue + émotions, etc.).

- Bilan carbone : émissions CO2 totales du projet mesurées par CodeCarbon.
- Roadmap : les versions du pipeline (V1 à V9) avec leurs objectifs et résultats.
- Conformité : fiches RGPD et IA Act résumant les mesures de conformité.

7. Utiliser le pipeline en Python

Le pipeline peut être utilisé directement en Python sans le dashboard :

```
import sys
sys.path.insert(0, 'src')

from pipeline.expert_detector import ExpertFakeNewsDetector
import pandas as pd

# Charger le modele
detector = ExpertFakeNewsDetector(model_dir='models', use_emotions=True)
detector.load(suffix='expert_v5')

# Analyser des textes
textes = pd.Series([
    "Le CNRS publie une etude sur le climat.",
    "BREAKING: Secret labs control your mind with 5G!!!",
    "La BCE maintient ses taux directeurs.",
])

resultats = detector.predict(textes)
print(resultats[['text', 'language', 'prediction_label', 'ai_score_credibility']])
```

Charger les modèles V6 et V7

```
import joblib

# Charger les modeles V6 et V7
v6_data = joblib.load('models/model_style_v6.joblib')
v7_data = joblib.load('models/model_hybrid_v7.joblib')
```

Colonnes retournées par predict() :

Colonne	Description
text	Texte original
language	Langue détectée (fr, en, other)
prediction_label	0 = Fiable, 1 = Suspect
ai_score_credibility	Probabilité de fiabilité (0 à 1)
ai_analysis_log	Log d'analyse lisible (ex: "[FR] Fiable (crédibilité: 89%)")

8. Utiliser l'analyse d'émotions

```
from pipeline.expert_detector import EmotionFeatureExtractor

emo = EmotionFeatureExtractor(model_dir='models')
```

```
emo.load()

probas = emo.get_emotion_features([
    "Je suis tellement heureux de cette nouvelle !",
    "This is absolutely terrifying news.",
])

# probas.shape = (2, 7) - 7 probabilités par texte
# Classes : colere, degout, joie, neutre, peur, surprise, tristesse
print(probas)
```

9. Notebooks

Les notebooks documentent chaque étape du projet et sont exécutés séquentiellement :

Notebook	Description	Sortie
00	Audit qualité des données d'entraînement	Statistiques, doublons, distributions
01	Exploration des posts Bluesky collectés	Visualisations, wordclouds, distributions temporelles
02	Entraînement du MLP émotions bilingue (PyTorch)	emotion_bilingual.pt + vocabulaire + label encoder
03	Pipeline de mise à jour quotidienne	Collecte + inférence sur posts récents
04	Prototype RoBERTa avancé	Exploration fine-tuning (non déployé)
05	Pipeline expert bilingue + ablation study (7 conditions)	model_expert.pkl + tfidf_expert.pkl + métriques
06	Documentation technique complète	Limites, roadmap, PRA/PCA, Green IT, conformité
09-24	Scripts d'entraînement V3-V7	Modèles V3 à V7, features stylistiques, méta-learner, SHAP
25	V8 CamemBERT intégration	Méta-learner étendu avec CamemBERT
26	Self-training Bluesky (échec)	Tentative de domain adaptation, échec documenté
27	Pipeline 2 étapes V9	Filtre fait/opinion + cascade V5, FP -67%

Pour ré-exécuter un notebook :

```
jupyter nbconvert --to notebook --execute notebooks/05_Detection_Expert_Bilingue.ipynb \
--output 05_Detection_Expert_Bilingue.ipynb --ExecutePreprocessor.timeout=600
```

10. Déploiement Docker

Le projet inclut un docker-compose.yml pour le déploiement complet :

```
# Lancer l'ensemble de la stack
docker-compose up -d

# Verifier les services
docker-compose ps

# Consulter les logs
docker-compose logs -f
```

Services :

- MongoDB : stockage des posts collectés et des résultats d'analyse.
- Collecteur Bluesky : script de collecte automatisée via le protocole AT.
- Dashboard Streamlit : interface web de visualisation (port 8501).
- Jupyter Notebook : serveur Jupyter pour l'exécution des notebooks d'analyse.

11. Configuration

Variables d'environnement (fichier .env)

```
BLUESKY_HANDLE=votre_handle.bsky.social
BLUESKY_APP_PASSWORD=votre_app_password
MONGODB_URI=mongodb://mongodb:27017/
```

Thème du dashboard (.streamlit/config.toml)

Le thème dark est configuré par défaut. Pour modifier les couleurs, éditez .streamlit/config.toml :

```
[theme]
primaryColor = "#00D4FF"
backgroundColor = "#0E1117"
secondaryBackgroundColor = "#1A1F2E"
textColor = "#E0E0E0"
```

12. FAQ

Q : Le dashboard affiche "Mode démo". Comment connecter MongoDB ?

R : Lancez MongoDB (via Docker ou en local sur le port 27017) et assurez-vous que la base thumalien_db contient une collection raw_posts. Le dashboard se connecte automatiquement au démarrage.

Q : Comment ré-entraîner le modèle avec de nouvelles données ?

R : Exécutez le notebook 05 (05_Detection_Expert_Bilingue.ipynb). Il recharge les CSV depuis data/training/, ré-entraîne le modèle et sauvegarde les fichiers .pkl dans models/.

Q : Le modèle classe mal un texte. Que faire ?

R : Le modèle est entraîné sur des articles de presse et peut mal généraliser sur des textes courts ou atypiques (posts de 10 mots, mèmes, satire). Le score doit être interprété comme un indicateur, pas comme un verdict. Consultez la section "Limites" du notebook 06.

Q : Puis-je ajouter d'autres langues ?

R : Le pipeline V9 supporte uniquement le français et l'anglais. Les textes dans d'autres langues sont routés vers le pipeline anglais par défaut.

Q : Qu'est-ce que le score SHAP affiche dans le dashboard ?

R : SHAP (SHapley Additive exPlanations) décompose la prédiction du modèle V6 en montrant la contribution de chaque feature stylistique. Une barre positive pousse vers "suspect", négative vers "fiable".

Q : Quel est le coût carbone d'une prédiction ?

R : Une prédiction sur un batch de 1 000 textes émet moins de 0.001 g de CO2. L'ensemble des entraînements du projet (LogReg + CamemBERT + RoBERTa) a émis 6.14 g de CO2, soit moins qu'une recherche Google (~7 g).

13. Support

- Code source : github.com/azelbanks/projet_etude
 - Documentation technique : notebook 06
 - Bugs et suggestions : ouvrir une issue sur le dépôt GitHub
-

Thumalien v9.0 - Pipeline bilingue FR/EN - Mastère Big Data, Sup de Vinci - Mai 2026